Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251172**
Nama Lengkap	**Ang, Joshua Alexander Hartono**
Minggu ke / Materi	**05 / Modular Programming**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Fungsi, Argument dan Parameter

Fungsi di Python adalah kumpulan kode yang dibuat untuk melakukan sebuah tugas tertentu dan dalam penggunaannya dapat digunakan berkali-kali. Contoh dari fungsi bawaan yang disediakan oleh Python adalah fungsi input() yang digunakan untuk menerima sebuah input / masukan dari pengguna dan fungsi print() untuk menampilkan hasil dari suatu proses ke layar. Adapun hubungan fungsi dengan modular programming adalah membantu membuat program menjadi lebih rapi serta, terstruktur, dan mudah dipahami terutama apabila kode program yang ditulis panjang. Dalam hal ini modular programming berperan dalam membuat program dengan membaginya ke dalam bagian-bagian kecil yang masing-masing memiliki fungsi tertentu.

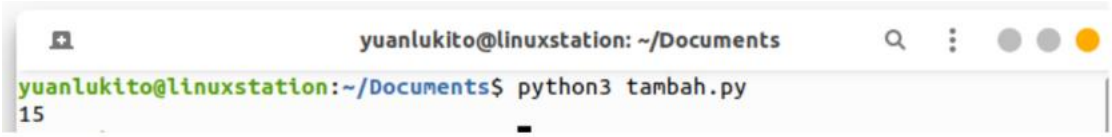
Berdasarkan asalnya, fungsi dibagi menjadi dua jenis yaitu fungi bawaan (built-in function) yang merupakan fungsi bawaan dari Python dan fungsi yang dibuat sendiri oleh programmer. Fungsi buatan sendiri oleh programmer ditunjukan dengan penulisan kata kunci def, yang kemudian diikuti oleh nama fungsi dan tanda kurung yang berisi parameter. Parameter adalah variabel yang digunakan untuk menampung nilai yang nantinya akan dikirim Kembali ke fungsi. Saat fungsi dipanggil, nilai yang dikirim tersebut disebut argument. Di dalam fungsi, programmer dapat menulis fungsi yang diinginkan sesuai dengan kebutuhan yang ada lalu menggunakan return untuk mengembalikan hasil dari proses fungsi.

```
1  # definisikan fungsi tambah terlebih dahulu
2  def tambah(a, b):
3      hasil = a + b
4      return hasil
5
6  # panggil fungsi tambah dengan dua arguments berupa nilai: 10 dan 5
7  c = tambah(10, 5)
8  print(c)
9
```

Gambar 5.1: Contoh penggunaan fungsi. *Sumber:* https://l1nq.com/EjWE2

`

Pada gambar di atas, fungsi ini didefinisikan dengan menggunakan keyword def. setelah mendefinisikan fungsi, kemudian kita menambahkan nama fungsi tambah() dengan tujuan untuk memudahkan dalam mengetahui peran dari fungsi tersebut yakni untuk menambahkan angka. Didalam nama fungsi kemudian kita menambahkan parameter berupa a dan b yang nantinya akan diinput sebuah angka yang akan di proses dan di tampung kedalam sebuah variabel dalam hal ini yaitu variabel hasil (menyimpan hasil dari penjumlahan variabel a dan b). setelahnya keyword return digunakan untuk mengembalikan nilai dari fungsi. Namun, fungsi ini tidak akan bekerja jika belum dipanggil. Dalam hal ini, pemanggilan fungsi diikuti dengan argumen yang sesuai dengan parameter yang telah dideklarasikan sebelumnya. Ketika variabel c yang menyimpan fungsi dipanggil dengan nilai 10 dan 5, maka nilai tersebut kemudian akan masuk ke parameter a dan b sesuai dengan urutannya. Selanjutnya perintah di dalam fungsi akan dijalankan yaitu menjumlahkan nilai a dan b, lalu menyimpan hasil di variabel hasil. Nilai yang ada di variabel hasil lalu dikembalikan ke program yang memanggil fungsi yaitu variabel c. dalam hal ini proses dapat diurutkan dalam baris menjadi 1-2-3-4-5-6-7-2-3-4-7-8.



Gambar 5.2: Hasil dari pemanggilan fungsi pada gambar 5.1. *Sumber:* https://l1nq.com/EjWE2

Return Value

Berdasarkan nilai yang dihasilkan, fungsi di Python secara umum dibagi menjadi dua jenis, yaitu fungsi yang mengembalikan nilai fungsi dan yang tidak mengembalikan nilai fungsi. Dalam hal ini fungsi yang tidak mengembalikan nilai disebut dengan void function. Fungsi ini biasanya hanya menjalankan perintah tertentu seperti menampilkan teks ke layar tanpa menampilkan nilai yang bisa dipakai lagi untuk proses selanjutnya. Contohnya adalah fungsi print_twice(), yang menerima satu parameter berupa message lalu menampilkan pesan tersebut sebanyak dua kali. Didalam fungsi ini tidak menggunakan return, sehingga tidak menghasikan nilai khusus. Jika

fungsi ini dipanggil dan dicetak maka Python akan menampilkan nilai None yang menandakan fungsi tidak mengembalikan nilai apapun.

```
def print_twice(message):
    print(message)
    print(message)

print_twice("Hello World!")
```

Gambar 5.3: Contoh penggunaan void function. *Sumber:* https://l1nq.com/EjWE2

```
Hello World!
Hello World!
None
```

Gambar 5.4: Hasil dari proses gambar 5.3. *Sumber:* https://l1nq.com/EjWE2

Berbeda dengan void function, fungsi yang mengembalikan nilai menggunakan keyword return. Fungsi ini memiliki peran untuk mengeluarkan nilai yang merupakan hasil dari fungsi dan mengakhiri fungsi. Sebagai contoh adalah fungsi tambah(a, b, c) yang menerima tiga parameter kemudian menjumlahkan ketiganya dan menyimpan hasilnya ke dalam variabel hasil. Lalu nilai dari variabel ini kemudian dikembalikan dengan menggunakan return.

```
1  def tambah(a, b, c):
2      hasil = a + b + c

3      return hasil
4
5  nilai1 = 70
6  nilai2 = 85
7  nilai3 = 55
8
9  rata_rata = tambah(nilai1, nilai2, nilai3)/3
10 print(rata_rata)
```

Gambar 5.5: Contoh fungsi yang membalikkan nilai. *Sumber:* https://l1nq.com/EjWE2

Optional Argument dan Named Argument

Dalam Python sebuah fungsi dapat memiliki parameter opsional, yaitu parameter yang sudah diberi nilai bawaan (default). Artinya, jika saat pemanggilan fungsi parameter tidak diisi, maka Python akan otomatis menggunakan nilai default yang sudah ditentukan. Contohnya pada fungsi

hitung_belanja(belanja, diskon=0), parameter diskon memiliki default 0 persen. Sehingga ketika fungsi dipanggil hanya dengan satu argument, fungsi akan tetap berjalan dan menganggap diskonnya 0%. Jika diskon diisi saat pemanggilan fungsi, maka nilai default tersebut akan berubah sesuai dengan nilai yang diberikan.

```
def hitung_belanja(belanja, diskon=0):
    bayar = belanja - (belanja * diskon)/100
    return bayar

print(hitung_belanja(100000))
print(hitung_belanja(100000, 10))
print(hitung_belanja(100000, 50))
```

Gambar 5.6: Contoh optional argument. *Sumber:* https://l1nq.com/EjWE2

Selain itu, Python juga mengenal yang namanya named argument, yaitu cara memanggil fungsi dengan menyebutkan nama parameternya secara langsung. Dengan ini, urutan pengisian tidak harus sama dengan urutan parameter fungsi. Misalkan pada fungsi cetak(a, b, c) bisa dipanggil dengan cara memasukkan cetak(a = 20, b=30, c =40) atau dengan mengacak urutannya seperti cetak(b=30, c=40, a =20).

```
def cetak(a, b, c):
    print("Nilai a: ",a)
    print("Nilai b: ",b)
    print("Nilai c: ",c)

cetak(a=20, b=30, c=40)
```

```
def cetak(a, b, c):
    print("Nilai a: ",a)
    print("Nilai b: ",b)
    print("Nilai c: ",c)

cetak(b=30, c=40, a=20)
```

Gambar 5.7: Contoh penggunaan named argument. *Sumber:* https://l1nq.com/EjWE2

```
Nilai a: 20
Nilai b: 30
Nilai c: 40
```

Gambar 5.8: hasil dari kedua named argument. *Sumber:* https://l1nq.com/EjWE2

Anonymous Function (Lambda)

Anonymous function adalah fungsi tanpa nama, fungsi ini digunakan untuk keperluan yang sederhana dan biasanya hanya dipakai di satu baris kode saja. Di Python, anonymous function didefinisikan menggunakan keyword lambda. Fitur ini merupakan fitur tambahan di Python dan berbeda dengan bahasa pemrograman seperti Haskell atau Lisp.

```
def tambah(a, b):
    hasil = a + b
    return hasil

print(tambah(10,20))
```

Gambar 5.9: bentuk fungsi menggunakan def. *Sumber:* https://l1nq.com/EjWE2

```
tambah = lambda a, b: a + b

print(tambah(10,20))
```

Gambar 5.10: bentuk fungsi menggunakan lambda. *Sumber:* https://l1nq.com/EjWE2

Secara umum, sebuah anonymous function terdiri dari tiga bagian utama, yaitu keyword berupa lambda, bound variabel atau parameter yang menerima nilai, dan body yang berisi ekspresi untuk menghasilkan nilai.

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : https://github.com/angjoshua-sudo/Prak-Alpro-2026.git

SOAL 1

Source code:

```python
def cek_angka(a1, a2 ,a3):
    if a1 == a2 or a2 == a3 or a1 == a3:
        return False
    elif a1 + a2 != a3 and a1 + a3 != a2 and a2 + a3 != a1:
        return False
    else:
        return True

print(cek_angka(1, 2, 3))
print(cek_angka(2, 2, 4))
print(cek_angka(1, 4, 6))
```

Penjelasan:

Program ini diawali dengan membuat sebuah fungsi bernama **cek_angka** yang berisi tiga parameter yaitu **a1, a2,** dan **a3**. Fungsi ini bertujuan untuk mengecek dua syarat utama. Syarat pertama yaitu ketiga parameter harus terdiri dari angka yang berbeda. Selanjutnya adalah ada kemungkinan untuk dua angka apabila dijumlahkan hasilnya sama dengan angka yang satunya.

Pada bagian **if**, program akan mengecek apakah ada angka yang nilainya sama dengan menggunakan operator **or**. Jika diantara parameter ditemukan ada parameter yang sama maka fungsi akan mengembalikan nilai **False** dan program selesai. Jika tidak ada yang sama maka program kemudian akan lanjut mengecek bagian elif. Pada bagian ini program akan mengecek apakah tidak ada dua angka yang jika dijumlahkan akan sama dengan angka yang lainnya. Hal ini dilakukan dengan menggunakan operator **!=** serta **and** yang jika semua kondisi benar fungsi akan mengembalikan nilai **False**. Terakhir jika kedua kondisi tersebut tidak terpenuhi maka program akan menjalankan bagian **else** dan mengembalikan nilai **True**. Setelahnya ada **print** yang berfungsi untuk menampilkan hasil dari fungsi yang dibuat.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu5/soal1.
py
True
False
False
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github>
```

SOAL 2

Source code:

```python
def cek_digit_belakang(d1, d2, d3):
    d1 = d1 % 10
    d2 = d2 % 10
    d3 = d3 % 10

    if d1 == d2 or d2 == d3 or d1 == d3:
        return True
    else:
        return False

print(cek_digit_belakang(30, 20, 18))
print(cek_digit_belakang(145, 5, 100))
print(cek_digit_belakang(71, 187, 18))
print(cek_digit_belakang(1024, 14, 94))
print(cek_digit_belakang(53, 8900, 658))
```

Penjelasan:

Program ini diawali dengan fungsi bernama **cek_digit_belakang** yang memiliki tiga parameter yaitu **d1, d2,** dan **d3**. Fungsi ini memiliki tujuan untuk mengecek apabila ada minimal dua dari tiga angka yang diberikan user memiliki digit terakhir yang sama. Di fungsi diawali dengan melakukan modulus pada setiap input yang diberikan user menggunakan modulus 10 (**%10**). Sehingga masing-masing variabel hanya menyisakan digit paling kanannya.

Setelah itu, program akan melakukan pengecekan kondisi **if**. Apabila saat pengecekan antara dua angka terdapat yang sama maka fungsi akan mengembalikan nilai **True**. Untuk proses kondisi ini disertai dengan penggunaan operator **or** dengan tujuan agar program dapat mengecek semua kemungkinan pasangan angka. Selama ada satu pasangan yang sama maka kondisi akan benar dan menghasilkan **True**. Namun, jika tidak ada yang sama maka fungsi akan menjalankan bagian **else** lalu mengembalikan nilai **False**. Setelahnya ada **print** yang berfungsi untuk menampilkan hasil dari fungsi yang dibuat.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu5/soal2.
py
True
True
False
True
False
```

SOAL 3

Source code:

```python
c_to_r = lambda c: 0.8 * c
c_to_f = lambda c: (9/5) * c +32

print(c_to_f(100))
print(c_to_r(80))
print(c_to_f(0))
```

Penjelasan:

Program diawali dengan pembuatan dua fungsi dengan tujuan untuk mengonversi nilai suhu menggunakan fungsi lambda. Pada baris pertama, fungsi **c_to_r** berfungsi untuk mengubah nilai suhu Celsius ke Reamur dengan rumus **R = 0.8 * C**. Jika ditulis dengan lambda menjadi lambda **c: 0.8 * c**. kemudian pada baris selanjutnya terdapat fungsi untuk mengubah nilai suhu dari Celsius ke Fahrenheit dengan rumus **F = (9/5) x C +32**. Yang kemudian ditulis menjadi lambda **c: (9/5) * c + 32**. Setelahnya terdapat **print** yang berguna untuk menampilkan hasil dari fungsi yang dibuat.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu5/soal3
py
212.0
64.0
32.0
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github>
```